STAT6011 tutorial1: R

Yiu Yin Yung

2025-09-04

R is a popular language used especially in the statistics area, and will be used throughout the course STAT6011. The detailed requirements can be found in Moodle.

This tutorial is a brief of the R language and the LaTeX. The target of this tutorial is to help you be able to finish your assignments smoothly, hence I try to make everything easy. If you are already quite familiar with R, feel free to attend/not attend this tutorial.

R

Firstly let me brief some important things about the Rstudio interface.

file -> new file -> R Markdown

Follow this way to create a new R Markdown file, or you may directly use the template provided in course Moodle (which is strongly suggested)

Coding R in R Markdown

To create an R chunk, click the following button.



There are some local options for the R chunk that can be added like you may see in the assignment template:

```
```{r, include=FALSE, eval=FALSE, echo=FALSE, warning=FALSE, message=FALSE,
tidy=TRUE}
```

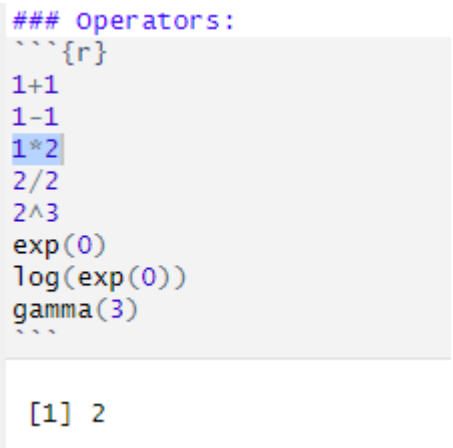
You can refer to the following table for their meanings:

Option	Run code	Show code	Output	Plots	Messages	Warnings
`eval = FALSE`	-		-	-	-	-
`include = FALSE`		-	-	-	-	-
`echo = FALSE`		-				
`results = "hide"`			-			
`fig.show = "hide"`				-		
`message = FALSE`					-	
`warning = FALSE`						-
```

To run all the code within a chunk, you can click the button at the right up corner of the chunk:



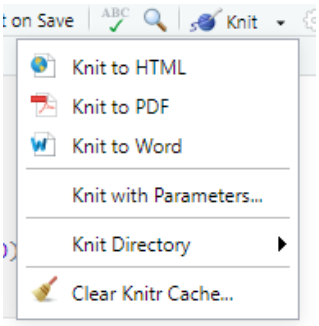
To run some selected code, you may use Ctrl+Enter:



```
### Operators:
```{r}
1+1
1-1
1*2
2/2
2^3
exp(0)
log(exp(0))
gamma(3)
```

[1] 2
```

After finishing your assignments, knit it to a .pdf file and submit both the .rmd codes and the .pdf file through Moodle:



Getting Help

1. In Rstudio
2. chatgpt (strongly suggested)

please write a rejection sampling algorithm in R for me

GPT-4o Poe

Certainly! Here's a simple rejection sampling algorithm in R:

```r
# Target distribution: Standard normal
target_density <- function(x) {
  dnorm(x)
}

# Proposal distribution: Uniform(-3, 3)
proposal_density <- function(x) {
  ifelse(x >= -3 & x <= 3, 1/6, 0)
}

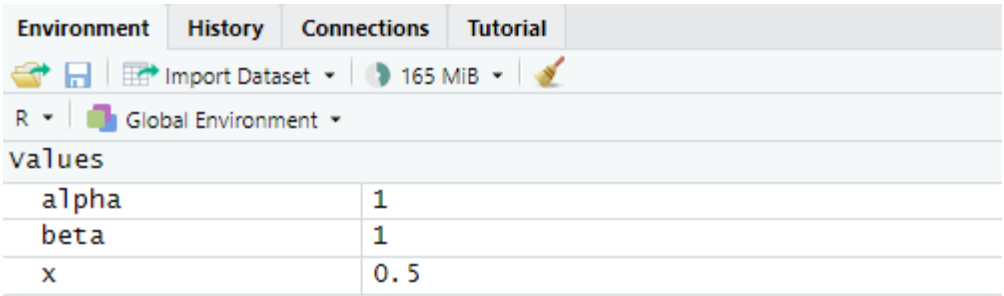
# Generate samples using rejection sampling
rejection_sampling <- function(n_samples) {
  samples <- numeric(n_samples)
  count <- 0

  while (count < n_samples) {
```

Copy

Environment

To check what we have in hand



R coding

Then, about the coding:

Data structure:

The data structures that are useful in assignments: vector, matrix and data frame

```
c(1,2,3,4)
```

```
## [1] 1 2 3 4
```

```r
c(1,2,3,4) %*% c(1,2,3,4)
```

```
##      [,1]
## [1,]   30
```

```r
matrix(c(1,2,3,4),nrow = 2,byrow = T)
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
```

```r
matrix(c(1,2,3,4),nrow = 2,byrow = T) %*%
  solve(matrix(c(1,2,3,4),nrow = 2,byrow = T))
```

```
##      [,1]         [,2]
## [1,]    1 1.110223e-16
## [2,]    0 1.000000e+00
```

```r
cbind(matrix(c(1,2,3,4),nrow = 2,byrow = T),matrix(c(1,2,3,4),nrow = 2,byrow = T))
```

```
##      [,1] [,2] [,3] [,4]
## [1,]    1    2    1    2
## [2,]    3    4    3    4
```

```r
rbind(matrix(c(1,2,3,4),nrow = 2,byrow = T),matrix(c(1,2,3,4),nrow = 2,byrow = T))
```

```
##      [,1] [,2]
## [1,]    1    2
## [2,]    3    4
## [3,]    1    2
## [4,]    3    4
```

```r
matrix(c(1,2,3,4),nrow = 2,byrow = T)[1,1:2]
```

```
## [1] 1 2
```

```r
df = data.frame(x = c(1,2,3,4), y=c(1,2,3,4))
mean(df$x)
```

```
## [1] 2.5
```

```r
df2 = readxl::read_xlsx("example.xlsx")
write.csv(df2, "example.csv")
df2 = read.csv("example.csv")
writexl::write_xlsx(df2,"example.xlsx")
```

Operators:

```r
1+1
```

```
## [1] 2
```

```r
1-1
```

```
## [1] 0
```

```r
1*2
```

```
## [1] 2
```

```r
2/2
```

```
## [1] 1
```

```r
2^3
```

```
## [1] 8
```

```r
exp(0)
```

```
## [1] 1
```

```r
log(exp(0))
```

```
## [1] 0
```

```r
factorial(4)
```

```
## [1] 24
```

```r
gamma(3)
```

```
## [1] 2
```

```r
for (i in 1:4){
  if (i<2){
    print(i)}
}
```

```
## [1] 1
```

For example, the Beta(α, β) likelihood is $\frac{\Gamma(\alpha+\beta)}{\Gamma(\alpha)\Gamma(\beta)} x^{\alpha-1}(1-x)^{\beta-1}$, suppose $x = 0.5$, then we may calculate it by:

```r
alpha_ = 1
beta_ = 3
x_ = 0.5
beta_density = function(x, alpha, beta){
  return((gamma(alpha+beta)/ (gamma(alpha) * gamma(beta))) *
           (x^(alpha-1)) * (1-x)^(beta-1))
}
beta_density(x_, alpha_, beta_)
```

```
## [1] 0.75
```

Useful package

For visualization, we recommend the package tidyverse, which is extremely useful in statistical analysis:

```r
#install.packages("tidyverse")
```

The package tidyverse is a strong tool for data cleaning, data analysis, and data visualization. So far we only focus on the visualization part, details can be found at https://ggplot2.tidyverse.org/

Load the package:

```r
library(tidyverse)
```

```
## Warning: package 'ggplot2' was built under R version 4.4.3
```

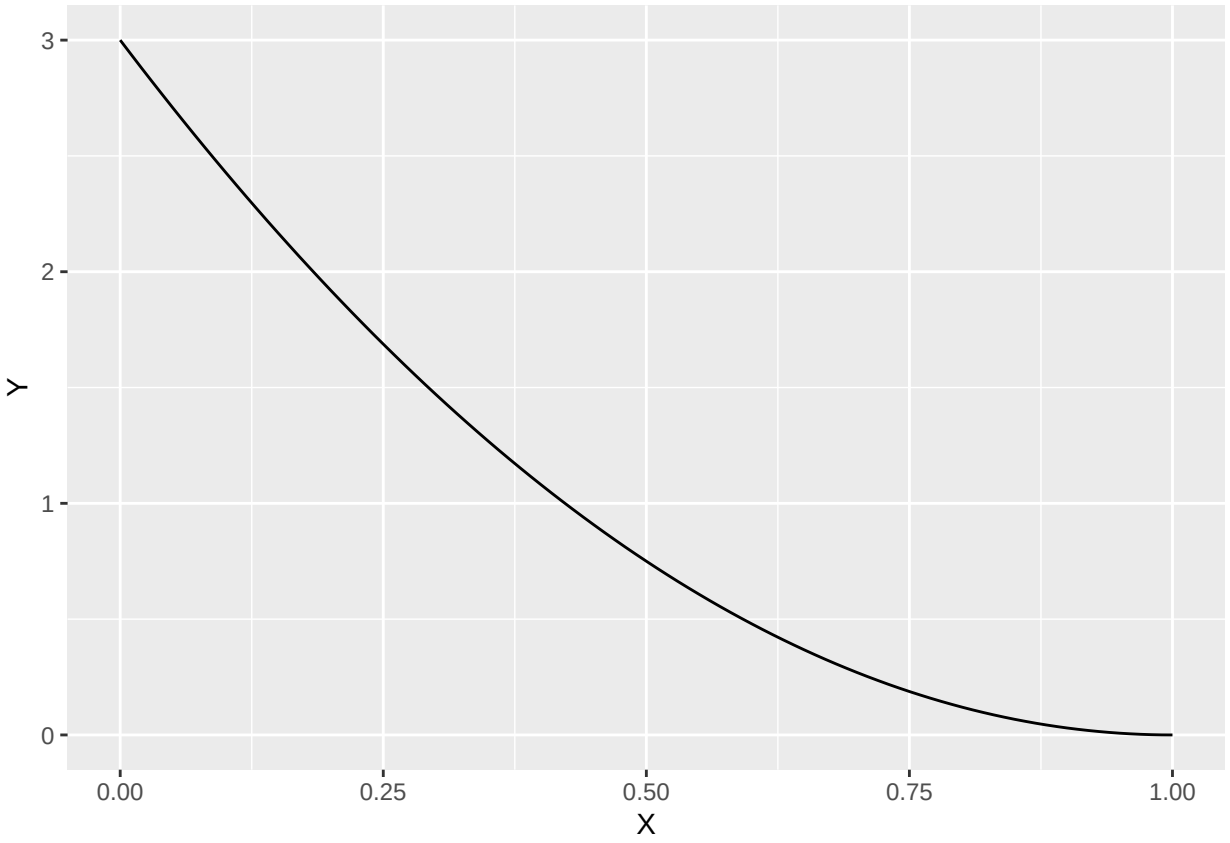
```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
```

```
## v dplyr     1.1.4     v readr     2.1.5
## v forcats   1.0.0     v stringr   1.5.1
## v ggplot2   4.0.2     v tibble    3.2.1
## v lubridate 1.9.4     v tidyr     1.3.1
## v purrr     1.0.4
## -- Conflicts ---------------------------------------- tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Density visualization

We can visualize the density as:

```
X = seq(from=0, to=1, by=0.01)
Y = beta_density(X, alpha_, beta_)
ggplot(mapping = aes(x=X, y=Y)) + geom_line()
```



Other useful functions

For commonly encountered distributions, there are some useful functions, take still the beta distribution as an example:

```
dbeta(x_, shape1=alpha_, shape2=beta_) #probability densiy
```

```
## [1] 0.75
```

```
pbeta(0.5, shape1=alpha_, shape2=beta_) #cumulative density
```

```
## [1] 0.875
```

```r
qbeta(0.875, shape1=alpha_, shape2=beta_) #quantile
```

```
## [1] 0.5
```

```r
rbeta(10, shape1 = alpha_, shape2=beta_) #generate a random sample
```

```
##  [1] 0.02365940 0.02703736 0.46388607 0.41525966 0.25554508 0.10893592
##  [7] 0.29866352 0.08461330 0.40500554 0.11052805
```

LaTeX

A useful language for typing math formulas. You can refer to the Mathematics section of the Overleaf documentation at https://www.overleaf.com/learn/latex/Mathematical_expressions for a detailed introduction.

How our professor prepares his slides:

For this section. if we don't have enough time in tutorial to cover, you may check with the lecture notes and run them by yourself, to have a better understanding of the lecture.

Monte Carlo Method:

Example: Monte Carlo Integration

- Suppose we compute $\int_0^1 h(x)dx$, where

$$h(x) = [\cos(50x) + \sin(20x)]^2.$$

- To calculate the integral, we generate $u_1, u_2, \ldots, u_n$ iid $\text{Unif}(0, 1)$ random variables, and approximate $\int_0^1 h(x)dx = \int_0^1 h(x) \cdot 1dx$ with $\sum_{i=1}^n h(u_i)/n$.
- It is clear that the Monte Carlo average is converging, with a value of 0.963 after 10,000 iterations.

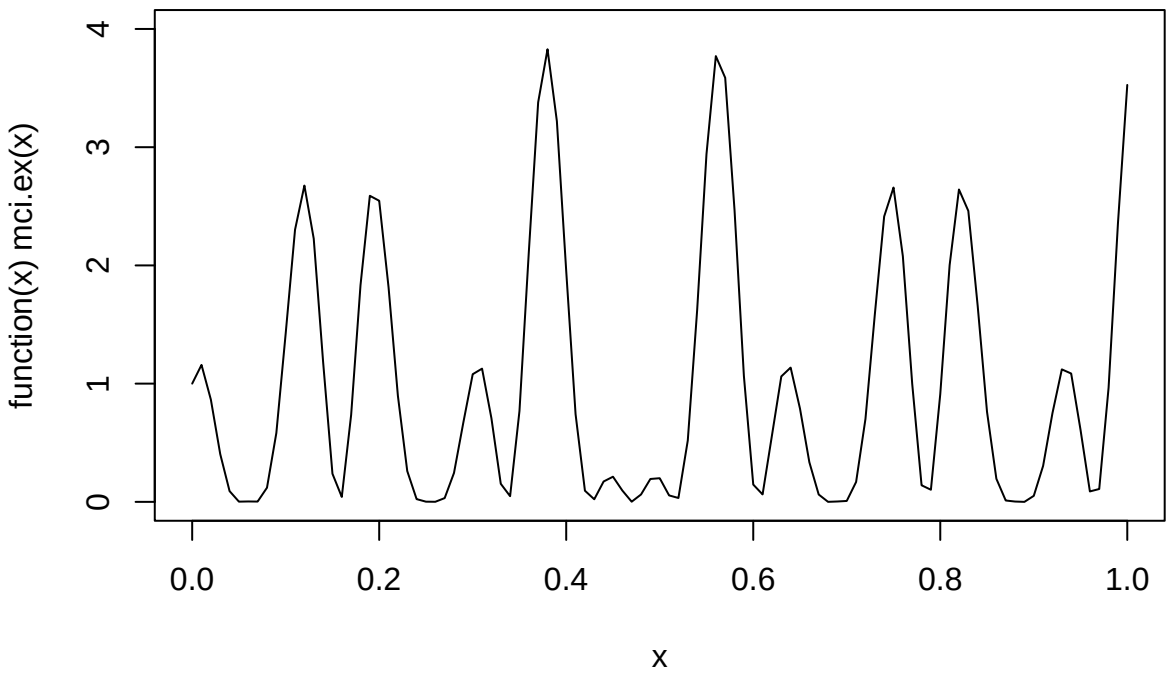
In the following example, we use the Monte Carlo method to calculate an integral $\int_0^1(\cos(50x) + sin(20x))^2dx$ from 0 to 1, where the samples are taken from a Uniform(0,1) distribution.

```r
# monteCarloMethod.R
# This code uses Monte Carlo method to compute the
# integration of f(x) from 0 to 1
#########################################################
rm(list=ls())   #to remove all the things in the environment
set.seed(1234)
a <- 0; b <- 1
nsim <- 10000000; u <- runif(nsim, a, b)

#The function to be integrated
mci.ex <- function(x)
```

```
  (cos(50*x)+sin(20*x))^2

plot(function(x)mci.ex(x), xlim=c(0,1),ylim=c(0,4))
```




```
#The monte carlo sum

nsim <- 10000000; u <- runif(nsim, a, b)
sum(mci.ex(u))/nsim
```

```
## [1] 0.9645119
```

- This idea can be extended to compute $\int_a^b h(x)dx$, where a and b are finite numbers,

$$\int_a^b h(x)dx = (b-a)\int_a^b h(x)\frac{1}{b-a}dx \approx (b-a)\sum_{i=1}^{n} h(u_i)/n,$$

  where u_i are random samples from $\mathrm{Unif}(a,b)$.

In the following example, the application of the Monte Carlo method has been extended, where the lower and upper bound can be any finite numbers a, b. We approximate the integral $\int_1^4 \sin x dx$.

```
# Another example: uses Monte Carlo method to compute the
# integration of f(x) from a to b (a, b can be any finite numbers)
#################################################################
rm(list=ls())
set.seed(1234)
```

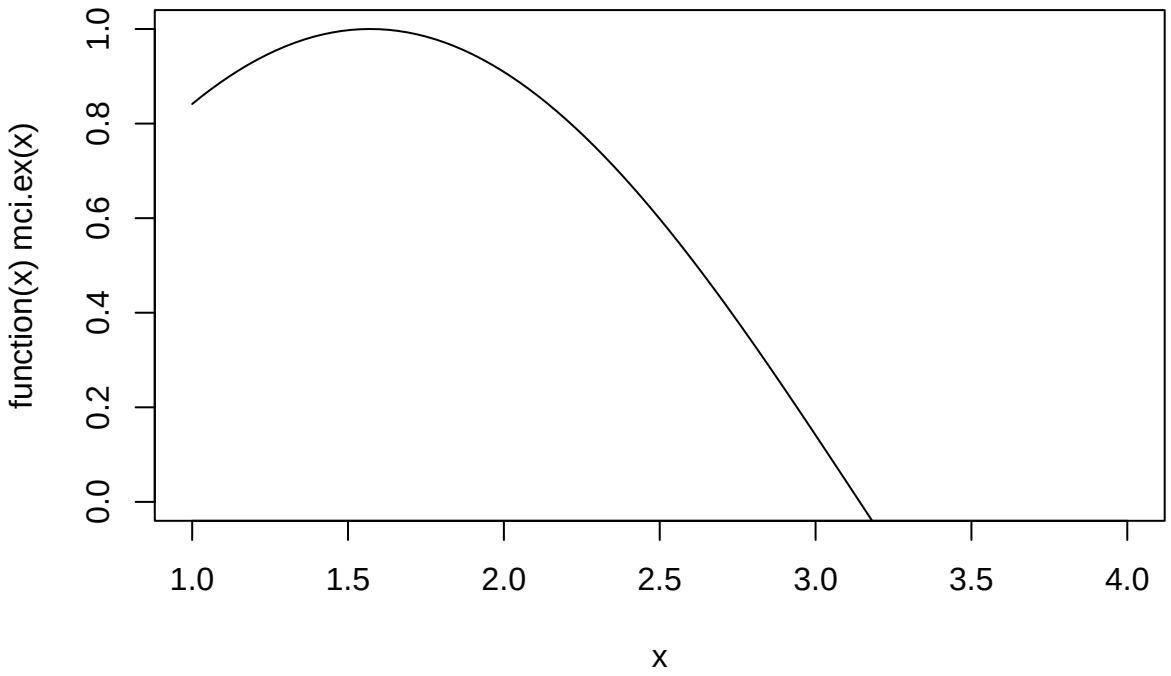
```r
a <- 1; b <- 4
nsim <- 1000000; u <- runif(nsim, a, b)

#The function to be integrated
mci.ex <- function(x) sin(x)

-(cos(b)-cos(a)) # 1.193946
```

```
## [1] 1.193946
```

```r
plot(function(x) mci.ex(x), xlim=c(a,b),ylim=c(0,1))
```




```r
#The monte carlo sum
(b-a)*sum(mci.ex(u))/nsim
```

```
## [1] 1.194514
```

Gamma to Beta

Transformation

- If $X \sim \text{Gamma}(\alpha, 1)$, and $Y \sim \text{Gamma}(\beta, 1)$, then

$$\frac{X}{X+Y} \sim \text{Beta}(\alpha, \beta).$$

To generate a random variable from $Beta(\alpha, \beta)$, we can generate $x \sim Gamma(\alpha, 1)$ and $y \sim Gamma(\beta, 1)$,

then $\frac{x}{x+y}$ follows our target Beta distribution.

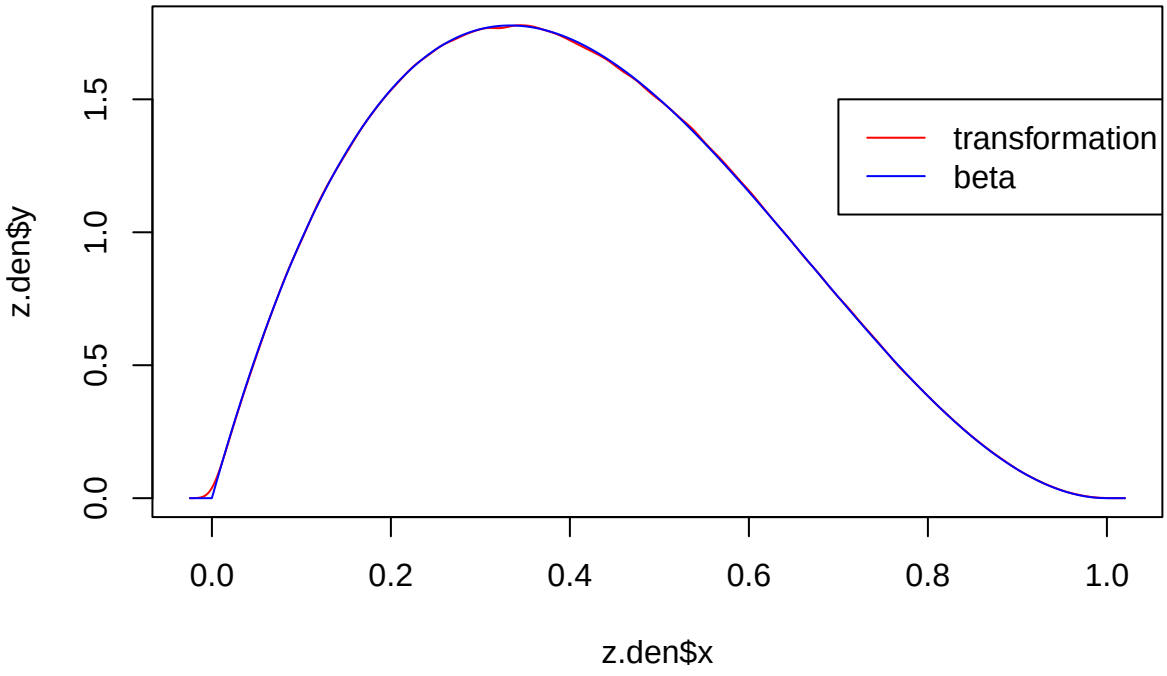
```
# gammaToBeta.R
# Simulate x from Gamma(alpha,1) and y from Gamma(beta,1)
# Then x/(x+y) has Beta(alpha, beta) distribution
###################################################################
rm(list=ls())
nSamples <- 5000000
alpha <- 2; beta <- 3

set.seed(1234)
x <- rgamma(nSamples, shape=alpha, rate=1)
y <- rgamma(nSamples, shape=beta, rate=1)

z <- x/(x+y)

z.den <- density(z)

plot(z.den$x, z.den$y, type="l", col="red")
points(z.den$x, dbeta(z.den$x,alpha,beta), type="l", col="blue")
legend(0.7, 1.5, legend=c("transformation","beta"),
       lty=c(1,1), col=c("red","blue"))
```




```
mean(z); var(z)
```

```
## [1] 0.4001003
```

```
## [1] 0.04001952
```

```
alpha/(alpha+beta)
```

```
## [1] 0.4
```

Rejection Sampling

- A more practical way to implement this is as follows:
  1. Simulate $u \sim \text{Unif}[0, 1]$ and $x \sim g(x)$.
  2. Accept x if $u \leq \frac{f(x)}{Cg(x)}$.
  3. Repeat steps 1 and 2 until we have enough simulated candidates.

In this example, the target distribution is $N(1, 4)$ and the proposal distribution is $Student - t(2)$ distribution.

```
# rejectionSampling.R
# In this example, I use rejection sampling methods to obtain samples for N(1,4) from
# t(df=2)
#########################################################################################
rm(list=ls())
set.seed(1234)

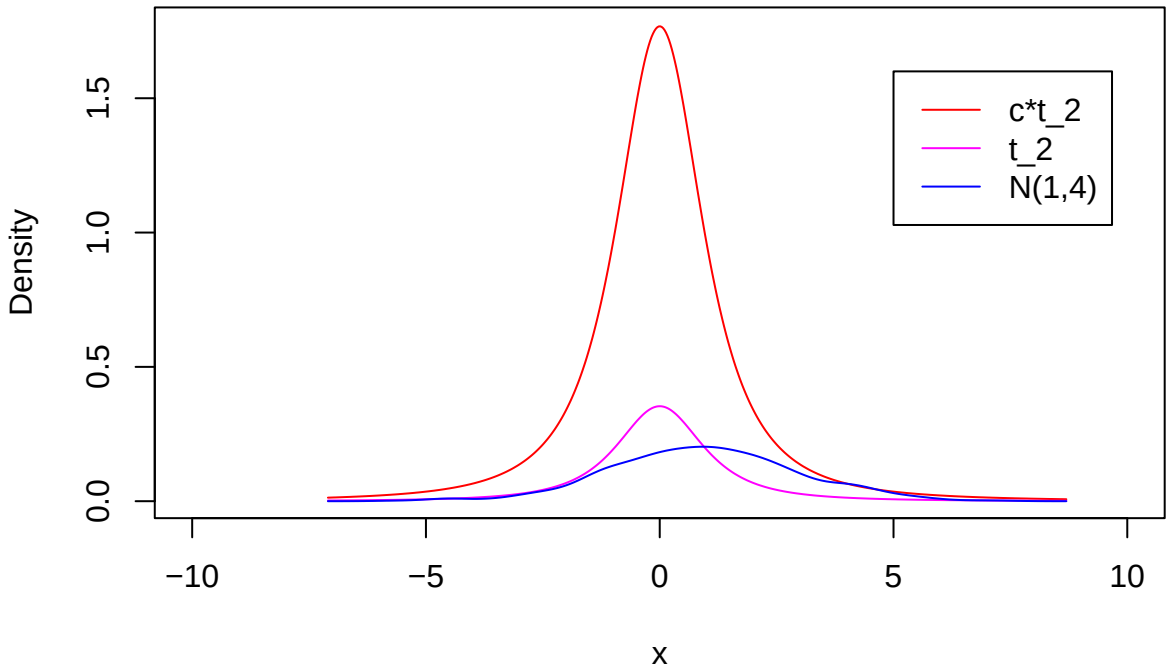
nsub <- 1000
myDf <- 2; myMean <- 1; mySd <- 2
x1 <- rnorm(nsub, mean = myMean, sd = mySd)
max(dnorm(x1, mean=myMean, sd=mySd)/dt(x1, df=2))
```

```
## [1] 4.990543
```

```
C <- 5

x1.den <- density(x1)

# plot two density function f(x) and Cg(x)
plot(x1.den$x, dt(x1.den$x,df=myDf)*C, xlim=c(-10,10), type="l", col="red",
     xlab="x", ylab="Density", main="Density function")
points(x1.den$x, dt(x1.den$x,df=myDf), type="l",col="magenta")
points(x1.den$x, x1.den$y, type="l", col="blue")
legend(5, 1.6, legend=c("c*t_2", "t_2", "N(1,4)"),
       lty=c(1,1,1), col=c("red","magenta","blue"))
```




```
# check if all C*t_2 points are smaller than target density
table(dnorm(x1.den$x, mean=myMean, sd=mySd)<=dt(x1.den$x,df=myDf)*C)
```

```
##
## TRUE
##  512
```

```
# rejection sampling starts from here
C <- 5
nsim <- 100000
samples.g <- rt(nsim, df=myDf)
accept <- runif(nsim) <= (dnorm(samples.g, mean=myMean, sd=mySd)
                          /(C*dt(samples.g,df=myDf)))
table(accept)/nsim
```

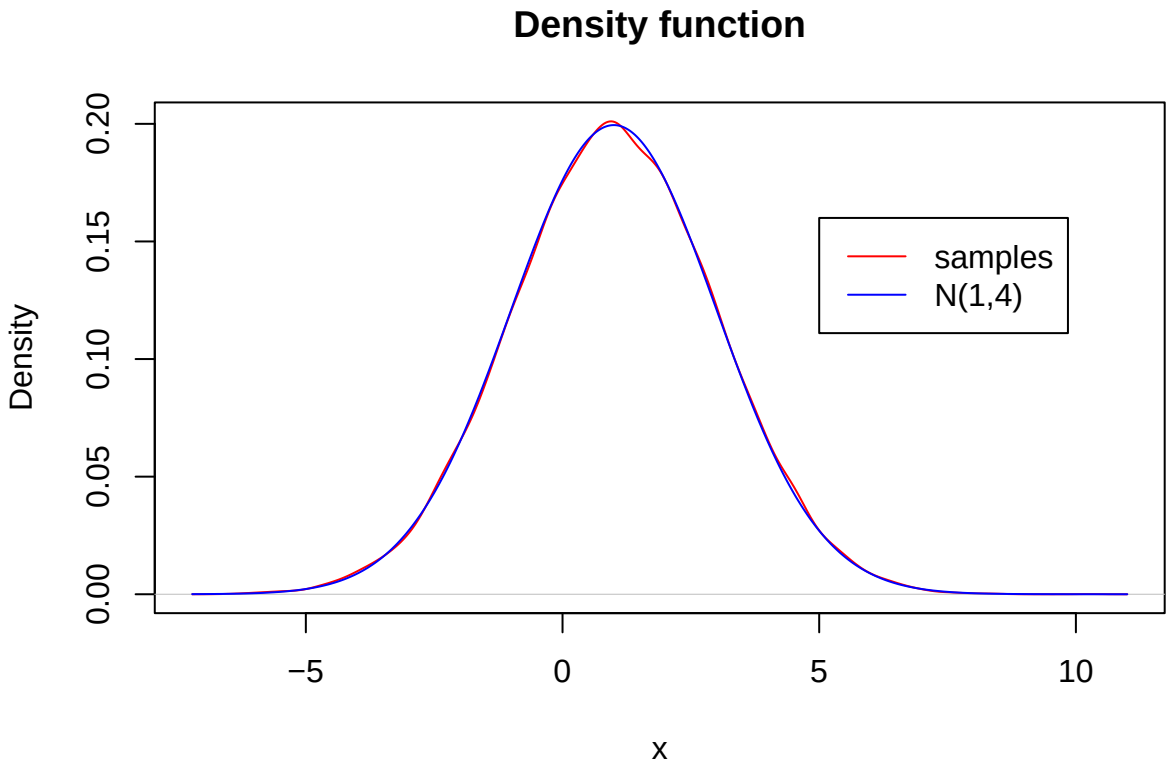
```
## accept
##   FALSE    TRUE
## 0.79857 0.20143
```

```
mySamples <- samples.g[accept]
summary(mySamples)
```

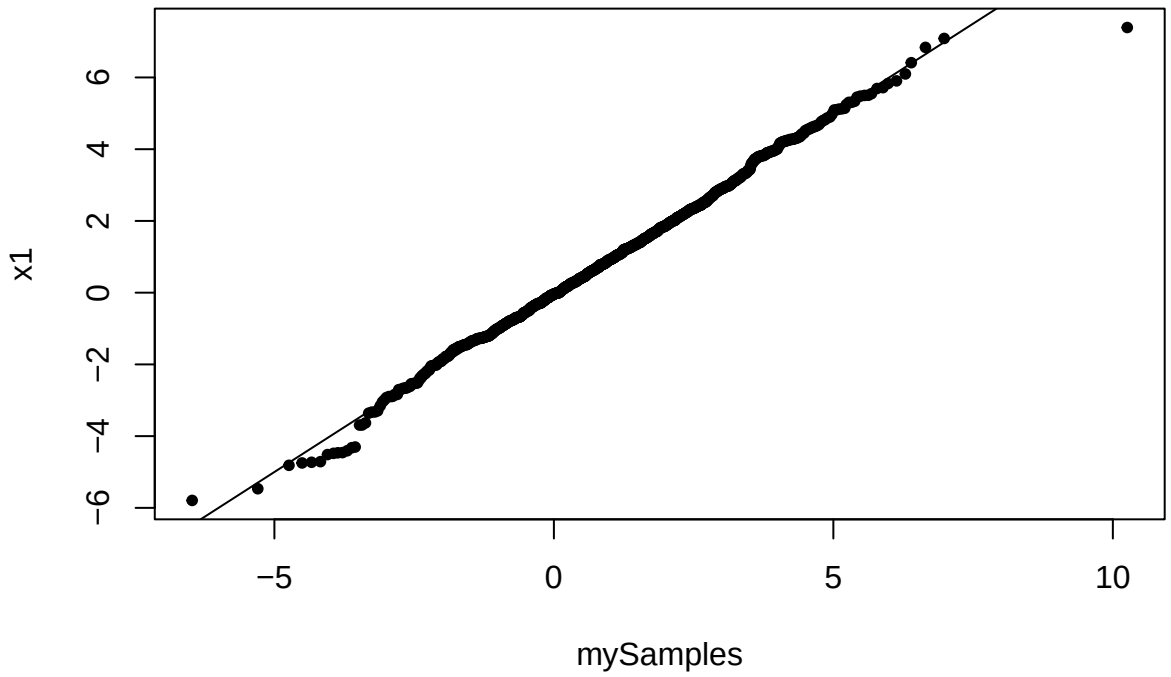
```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  -6.471  -0.326   1.007   1.009   2.362  10.258
```

```
mySamples.den <- density(mySamples)

# compare the density functions of the obtained samples and R samples
plot(mySamples.den, col="red", xlab="x", main="Density function")
points(mySamples.den$x, dnorm(mySamples.den$x, mean=myMean, sd=mySd), type="l", col="blue")
legend(5, 0.16, legend=c("samples", "N(1,4)"), lty=c(1,1), col=c("red","blue"))
```




```r
# Q-Q plot raw vs simulated densities
qqplot(mySamples, x1, pch=20)
abline(0, 1)
```



- Any constant, $C' > C$, will work. Because the acceptance rate is $\frac{1}{C'}$, C' should be as close to the optimal C as possible.

In the following example, we show that the constant C used can be larger than the optimum one.

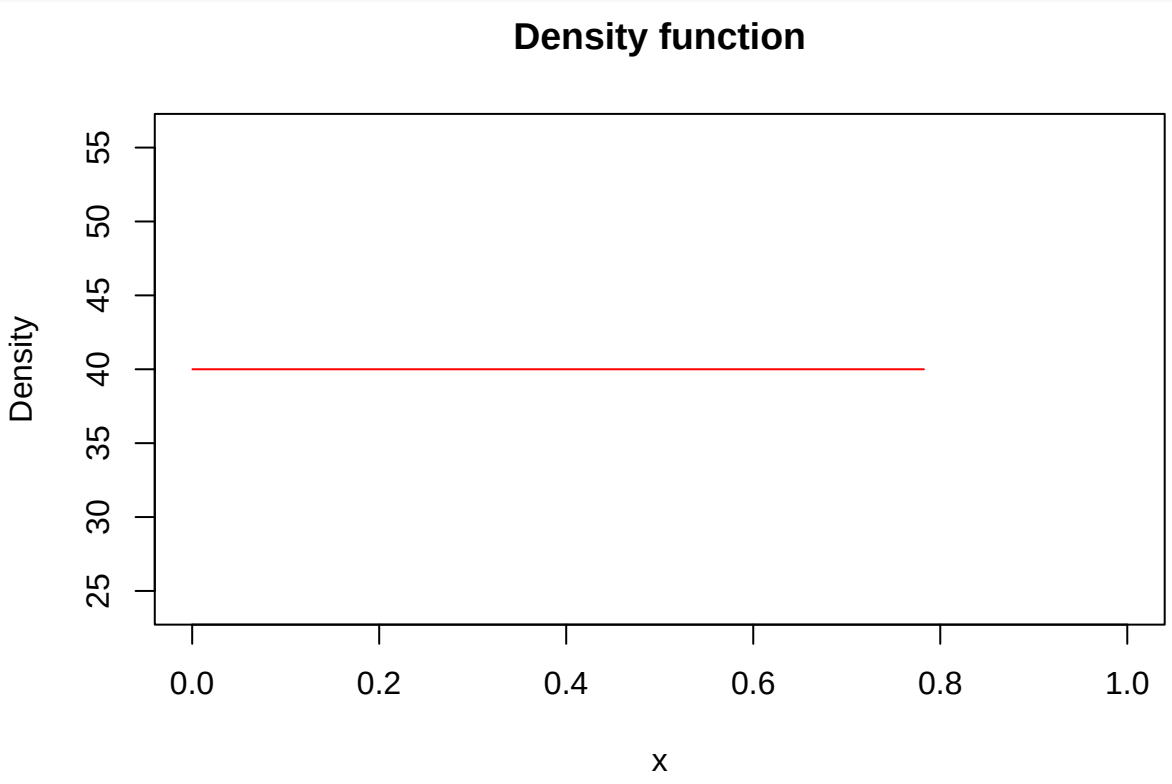
```
# 1. Define a density of interest that will be approximated by "REJECTION SAMPLING"
rm(list=ls())
knownDensity <- function(x) {
  return(dbeta(x, 3, 10))
}

sampleSize <- 3000
rawDensity <- rbeta(sampleSize, 3, 10)
rawDensity.den <- density(rawDensity)

rawDensity.x <- rawDensity.den$x[rawDensity.den$x >=0]
rawDensity.y <- rawDensity.den$y[rawDensity.den$x >=0]

C <- 40

# plot two density function f(x) and Cg(x)
plot(rawDensity.x, dunif(rawDensity.x)*C, xlim=c(0,1), type="l", col="red",
     xlab="x", ylab="Density", main="Density function")
points(rawDensity.x, rawDensity.y, type="l", col="blue")
legend(5, 1.6, legend=c("c*t_2", "target"),
       lty=c(1,1), col=c("red","blue"))
```




```
# 2. Rejection sampling method
xMin <- 0; xMax <- 1
mySequence <- seq(from=xMin, to=xMax, by=0.01)
max(knownDensity(mySequence))
```

```
## [1] 3.584376
```

```r
C <- 40


nsim <- 10000000
samples.g <- runif(nsim)
u <- runif(nsim)
accept <- u <= (knownDensity(samples.g)/(C*1))
table(accept)/nsim
```

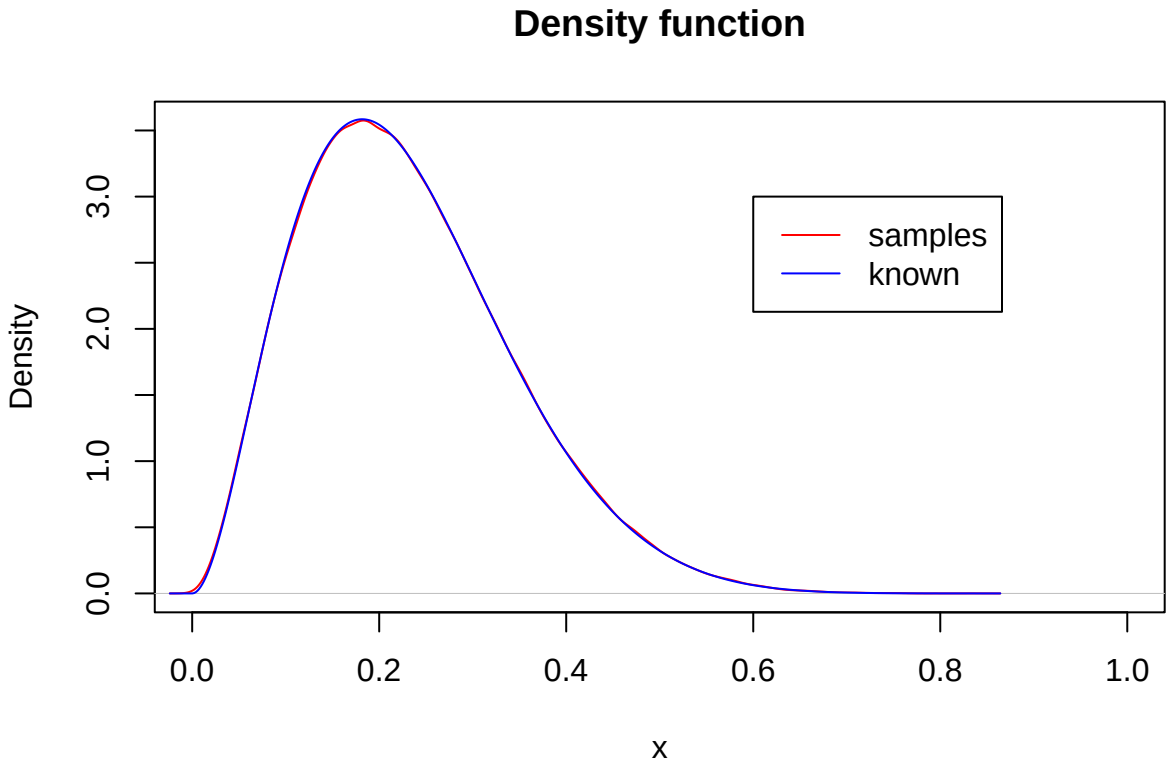
```
## accept
##     FALSE      TRUE
## 0.9749692 0.0250308
```

```r
mySamples <- samples.g[accept]
summary(mySamples)
```

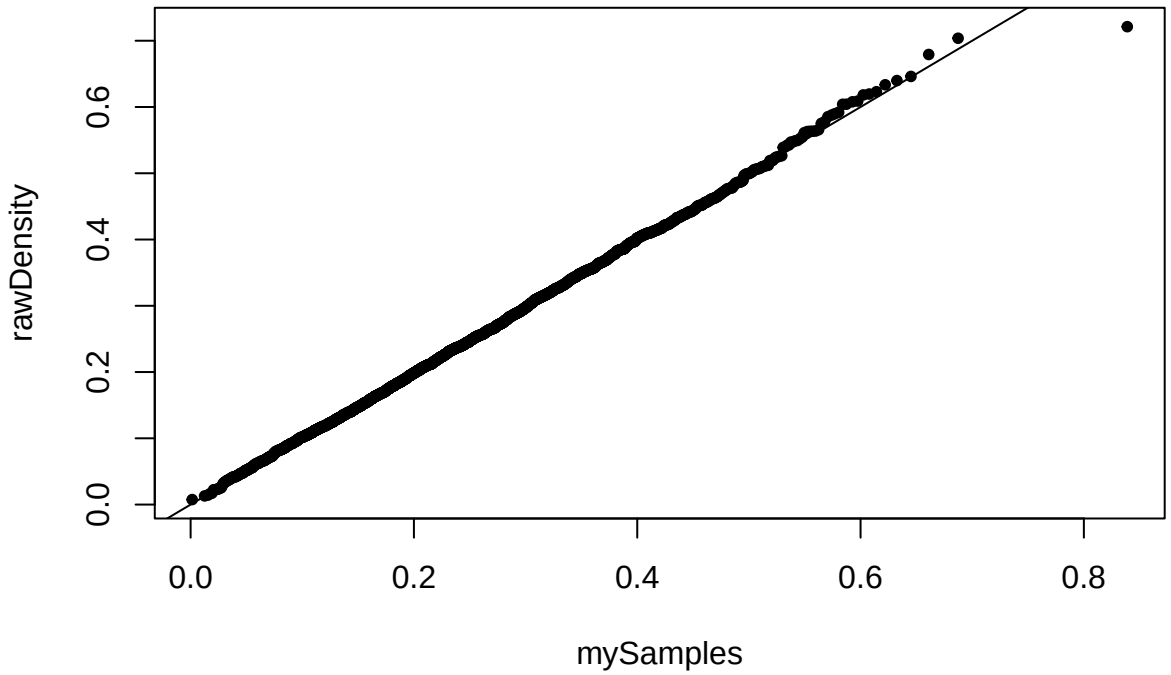
```
##     Min.  1st Qu.   Median     Mean  3rd Qu.     Max.
## 0.001403 0.145943 0.216895 0.230980 0.301498 0.838886
```

```r
mySamples.den <- density(mySamples)

# plots of raw vs simulated density function
plot(mySamples.den, col="red", xlim=c(0,1), xlab="x", main="Density function")
points(mySamples.den$x, knownDensity(mySamples.den$x), type="l", col="blue")
legend(0.6, 3, legend=c("samples", "known"), lty=c(1,1), col=c("red","blue"))
```




```r
# Q-Q plot raw vs simulated densities
qqplot(mySamples, rawDensity, pch=20)
abline(0, 1)
```



Example

- Suppose $y_1, \ldots, y_n \sim N(\mu, \psi)$. Let $\boldsymbol{\theta} = (\mu, \psi)$. We define the priors: $\mu \sim N(0, \sigma^2)$ and $\psi \sim \text{IG}(\alpha, \beta)$.
- Note that $\hat{\mu} = \bar{y}$ and $\hat{\psi} = \sum_{i=1}^n (y_i - \bar{y})^2/n$.
- So, the optimal value is

$$C^* = f(\boldsymbol{y}|\hat{\mu}, \hat{\psi}) = \left(\frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n} \right)^{-n/2} e^{-\frac{n}{2}}.$$

- We simulate $u \sim \text{Unif}[0, 1]$, $\mu \sim N(0, \sigma^2)$, $\frac{1}{\psi} \sim \text{Gamma}(\alpha, \beta)$.

- We accept (μ, ψ) if

$$u \leq \frac{f(\boldsymbol{y}|\mu, \psi)}{f(\boldsymbol{y}|\hat{\mu}, \hat{\psi})} = \left(\frac{\sum_{i=1}^n (y_i - \bar{y})^2}{n\psi} \right)^{\frac{n}{2}} \exp\left\{ -\frac{\sum_{i=1}^n (y_i - \mu)^2}{2\psi} + \frac{n}{2} \right\}.$$

In this example, the target distribution is the posterior while the proposal distribution is the prior, where the rejection probability is proportional to the likelihood.

```r
# Example of rejection sampling from class
rm(list=ls())
set.seed(1234)
myMu <- 1; mySigma2 <- 4
n <- 150
y <- rnorm(n, mean=myMu, sd=sqrt(mySigma2))
ybar <- mean(y)
c(mean(y), var(y))
```

```
## [1] 0.8043031 3.6858004
```

```r
C.star <- (sum((y-ybar)^2)/n)^(-n/2)*exp(-n/2)

sim <- 1000000

#sigma <- 1; alpha <- 2; beta <- 1
sigma <- 10; alpha <- 0.01; beta <- 0.01
# simulate some candidates
mu <- rnorm(sim, mean = 0, sd = sigma)
psi <- 1 / rgamma(sim, shape=alpha, rate=beta)

# The acceptance probabilities on the log scale
log.fbycg <- (0.5*n*log(sum((y-ybar)^2)/n/psi)
              - (sum(y^2)-2*sum(y)*mu+n*mu^2)/2/psi + n/2)
str(log.fbycg)
```

```
##  num [1:1000000] -1624 -15357 -7863 -936 -6065 ...
```

```r
# a boolean vector, TRUE at the indices where we accepted
accept <- log(runif(sim)) <= log.fbycg
table(accept)/sim
```

```
## accept
##    FALSE     TRUE
## 0.999963 0.000037
```

```r
# the acceptance probability
mean(accept)
```

```
## [1] 3.7e-05
```

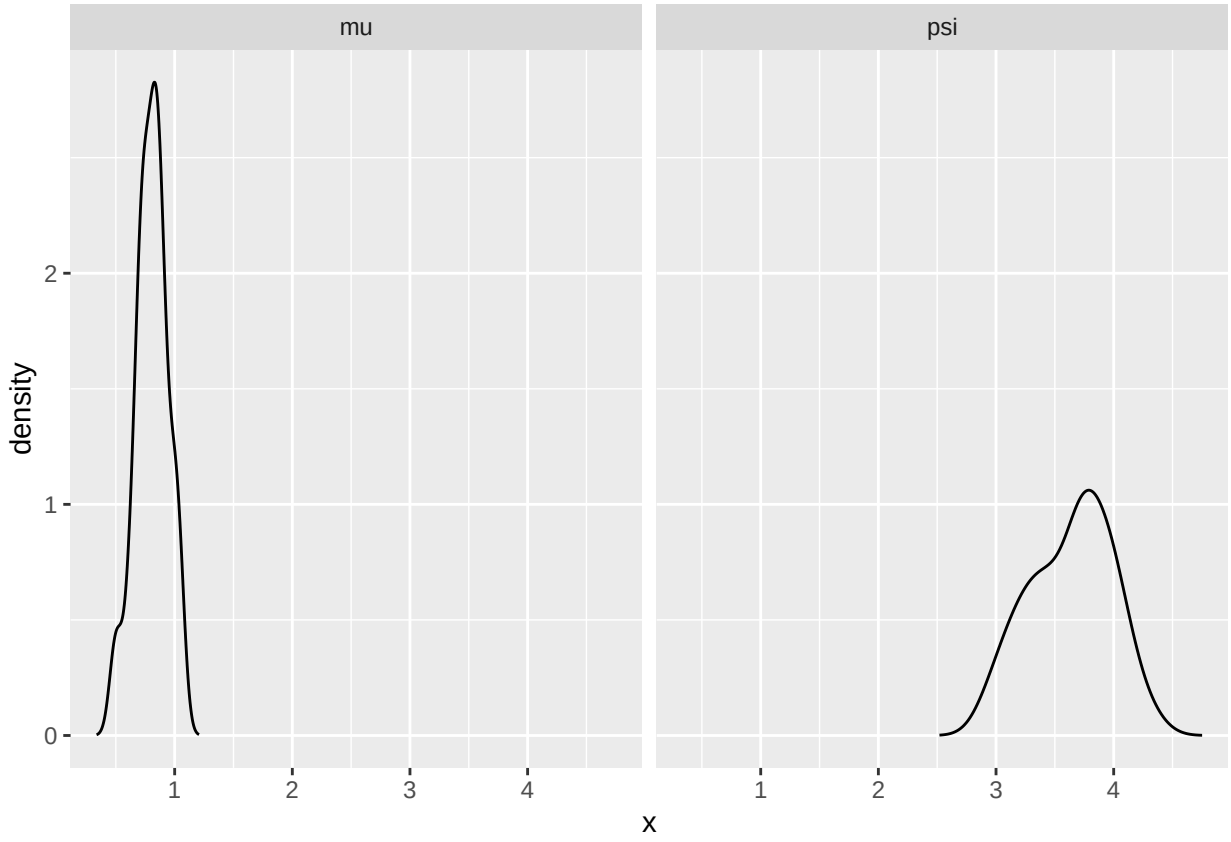
```r
# the accepted mu's and psi's
mu <- mu[accept]
psi <- psi[accept]

c(mean(mu), mean(psi))
```

```
## [1] 0.8054366 3.6357528
```

```r
# plot the histograms and density estimates
#postscript("./plots/rejectionSampling_posterior.eps", paper="letter", horizontal=T)
#par(mfrow=c(2,2))
 # hist(mu)
  #plot(density(mu))
  #hist(psi)
  #plot(density(psi))
#dev.off()
```

```
df = data.frame(rbind(cbind(density(mu)$x, density(mu)$y, rep("mu",length(density(mu)$x))),
           cbind(density(psi)$x, density(psi)$y, rep("psi",length(density(psi)$x)))))
names(df) = c("x", "density", "parameter")
df$x = as.numeric(df$x)
df$density = as.numeric(df$density)
ggplot(data=df)+geom_line(mapping = aes(x, density))+ facet_wrap(~ parameter, nrow = 1)
```




```
# equi-tail Bayesian confidence intervals
quantile(mu, c(.025,.975))
```

```
##      2.5%     97.5%
## 0.5076655 1.0408841
```

```
quantile(psi, c(.025,.975))
```

```
##     2.5%    97.5%
## 2.979674 4.133953
```

```
# posterior mean
mean(mu)
```

```
## [1] 0.8054366
```

Importance Sampling

- The *non-ratio version* of importance sampling is

$$E_f[h(x)] = E_g\left[\frac{f(x)}{g(x)}h(x)\right] \approx \frac{1}{n}\sum_{i=1}^{n}\frac{f(x_i)}{g(x_i)}h(x_i) = \frac{1}{n}\sum_{i=1}^{n} w_i h(x_i),$$

  where $x_1, \ldots, x_n$ are samples from $g(x)$, $w_i = \frac{f(x_i)}{g(x_i)}$ is called the *importance ratio* or *importance weight*.

- Note that

$$1 = E_f[1] = E_g\left[\frac{f(x)}{g(x)}\right] \approx \frac{1}{n}\sum_{i=1}^{n} w_i.$$

- The *ratio version* of importance sampling is

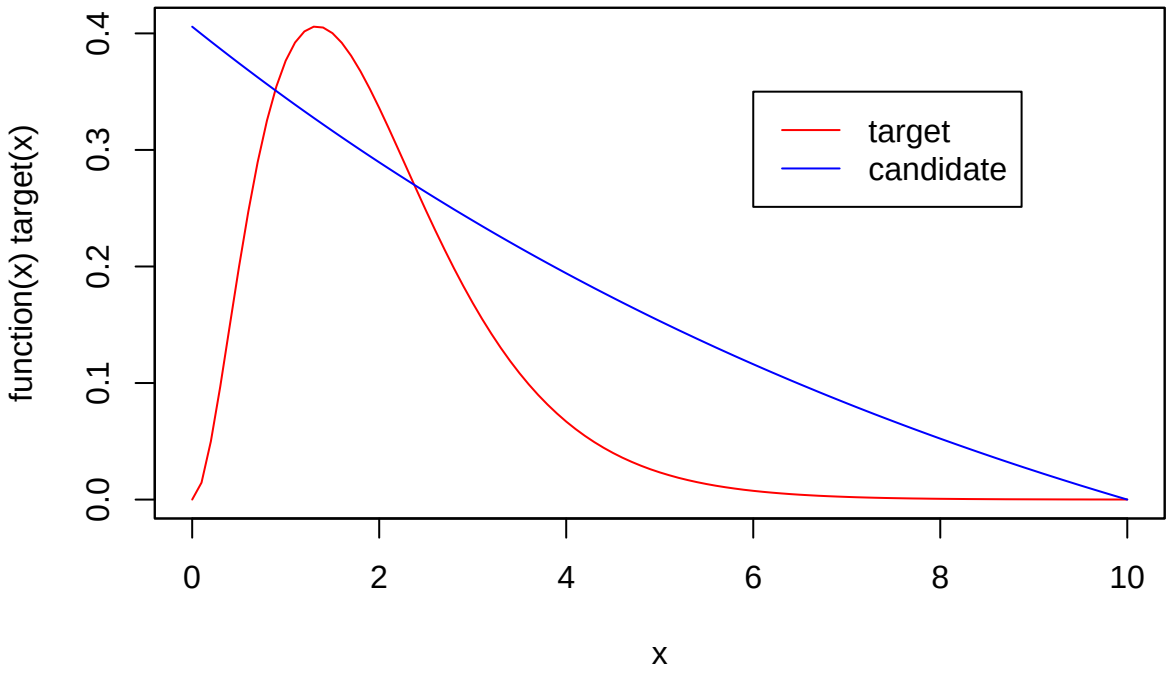
$$E_f[h(x)] = \frac{E_f[h(x)]}{E_f[1]} \approx \frac{\sum_{i=1}^{n} w_i h(x_i)}{\sum_{i=1}^{n} w_i}.$$

This example demonstrates how to computes the mean and variance of $Gamma(3, 2/3)$ (target distribution) from $Exponential(1)$ (proposal distribution) using both the ratio and non-ratio versions of importance sampling method.

```
# ImpSamp_gamma.R
# Compute the mean and variance of Gamma(3,2/3) from Exponential(1)
# True values: mean=2, var=1.33
#####################################################################

#target = Gamma(2,2/3), candidate = exponential
rm(list=ls())
nsim <- 10000
target <- function(x)((27/16)*(x^2)*exp(-3*x/2))
candidate <- function(x)(exp(-x))

plot(function(x)target(x),xlim=c(0,10), col="red")
legend(6,0.35, legend=c("target","candidate"),lty=c(1,1),col=c("red","blue"))
par(new=T)
plot(function(x)candidate(x),xlab="",ylab="",xaxt="n",yaxt="n",col="blue")
```




```
#Compute the mean and variance using Importance Sampling in non-ratio version
y <- rexp(nsim)
m1 <- sum(y*target(y)/candidate(y))/nsim
m2 <- sum((y^2)*target(y)/candidate(y))/nsim
m1;  m2-m1^2
```

```
## [1] 1.933809
```

```
## [1] 1.296763
```

```
#Compute the mean and variance using Importance Sampling in ratio version
m1 <- sum(y*target(y)/candidate(y))/sum(target(y)/candidate(y))
m2 <- sum((y^2)*target(y)/candidate(y))/sum(target(y)/candidate(y))
m1;  m2-m1^2
```

```
## [1] 1.966346
```

```
## [1] 1.254603
```

Ex: Importance Sampling for Sensitivity Analysis

- We have i.i.d. samples $y_i \sim f(y|\theta)$, for $i = 1, \ldots, n$, and prior $\pi(\theta|\psi_0)$.
- Suppose we have worked really hard to obtain a sample $\{\theta_i\}_{i=1}^n$ from $\pi(\theta|\boldsymbol{y}, \psi_0)$ and we aim to calculate $E[h(\theta)|\boldsymbol{y}, \psi]$ for values of the hyperparameter different from ψ_0.
- Do we have to re-simulate? No, we can just *importance reweigh* the samples we already have, i.e., $f = \pi(\theta|\boldsymbol{y}, \psi)$ and $g = \pi(\theta|\boldsymbol{y}, \psi_0)$. Then with cancelation of the likelihood,

$$E_f[h(\theta)|\boldsymbol{y}, \psi] \approx \frac{\sum_{i=1}^n h(\theta_i)\frac{\pi(\theta_i|\boldsymbol{y},\psi)}{\pi(\theta_i|\boldsymbol{y},\psi_0)}}{\sum_{i=1}^n \frac{\pi(\theta_i|\boldsymbol{y},\psi)}{\pi(\theta_i|\boldsymbol{y},\psi_0)}} = \frac{\sum_{i=1}^n h(\theta_i)\frac{\pi(\theta_i|\psi)}{\pi(\theta_i|\psi_0)}}{\sum_{i=1}^n \frac{\pi(\theta_i|\psi)}{\pi(\theta_i|\psi_0)}}$$

The 100 observations y_i are actually *i.i.d.* from $\mathcal{N}(1, 2)$. Well. let's pretend that we don't know this thing, we only know that y_i from certain $\mathcal{N}(\mu, \sigma^2)$ distribution, and we are interested in μ now. We put a $\mathcal{N}(\alpha, \beta)$ prior over μ. Our proposal distribution is hence the posterior distribution under the condition $\alpha = -5$, while our target is the posterior distribution under the condition $\alpha = 5$.

```
# ImpSamp_sensitivity.R
# Using importance sampling for sensitivity analysis
########################################################################

rm(list=ls())

# myMu, sigma: parameters for the data y_i
myMu <- 1; sigma <- 2
# n: number of observations
n <- 100
# beta: variance of the prior distribution of the mean of y_i
beta <- 1000000

set.seed(1234)
# y: observations
y <- rnorm(n, mean=myMu, sd=sqrt(sigma))
ybar <- mean(y)

# alpha0, alpha1: hyperparameters
alpha0 <- -5; alpha1 <- 5

# the posterior mean
cal.mean <- function(alpha, beta) {
  (ybar*beta+alpha*sigma/n)/(beta+sigma/n)
}

# the posterior variance
cal.var <- function(beta) {
```

```
  (beta*sigma/n)/(beta+sigma/n)
}

# nSamples: number of samples from posterior distribution of \mu
#           with hyperparameters \alpha_0 and \beta
nSamples <- 1000000
# mySamples: samples from posterior distribution of \mu
#           with hyperparameters \alpha_0 and \beta
mySamples <- rnorm(nSamples, mean=cal.mean(alpha0,beta),
                   sd=sqrt(cal.var(beta)))
summary(mySamples)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  0.0932  0.6827  0.7784  0.7784  0.8738  1.4713
```

```
# den.new: the density function of the samples
#          under the hyper-parameters \alpha_1 and \beta
den.new <- dnorm(mySamples, mean=cal.mean(alpha1,beta),
                 sd=sqrt(cal.var(beta)))
# den.old: the density function of the samples
#          under the hyper-parameters \alpha_0 and \beta
den.old <- dnorm(mySamples, mean=cal.mean(alpha0,beta),
                 sd=sqrt(cal.var(beta)))

h <- function(x) x^2

# importance sampling
sum(h(mySamples)*den.new/den.old)/sum(den.new/den.old)
```

```
## [1] 0.6258823
```

```
sum(h(mySamples)*den.new/den.old)/nSamples
```

```
## [1] 0.6258823
```

```
# true values
true.value <- cal.var(beta) + (cal.mean(alpha1,beta))^2
true.value
```

```
## [1] 0.6257595
```

Example: Normal Tail Probabilities

- For $a = 3.5, 4.5, 5.5$, calculate $P(Z > a) = \int_a^\infty \phi(x)dx$, where $Z \sim N(0,1)$.
- *"Naive" approach*: $x_i \sim N(0,1)$, $\int_a^\infty \phi(x)dx = E[I(x > a)]$. So

$$\frac{1}{n}\sum_{i=1}^{n} I(x_i > a) \to \int_a^\infty \phi(x)dx.$$

- *Importance sampling*: $x_i \sim g(x) = ae^{-a(x-a)}$, where $x > a$. Then $\int_a^\infty \phi(x)dx = \int_a^\infty \left[\frac{\phi(x)}{g(x)}\right] g(x)dx$. So,

$$\frac{1}{n}\sum_{i=1}^{n} \frac{\phi(x_i)}{ae^{-a(x_i-a)}} \to \int_a^\infty \phi(x)dx.$$

- *Transform to Uniform*

$$\int_a^\infty \phi(x)dx = \int_0^{1/a} \frac{\phi(1/y)}{y^2}dy, \quad \text{let } y = 1/x.$$

For $u_i \sim \text{Unif}(0, 1/a)$ with density $g(x) = a$,

$$\frac{1}{n}\sum_{i=1}^{n} \frac{\phi(1/u_i)}{au_i^2} \to \int_a^\infty \phi(x)dx.$$

```
#This gives an importance sampling algorithm for a Truncated Normal#
a <- 3.5                    #Truncation Point
n <- 500000              #Sample size
#---------Note!!  R can do this with 1-pnorm(3.5) and 1-pnorm(4.5)
1-pnorm(a)
```

```
## [1] 0.0002326291
```

```
#---------Generate a Sample Naively-------------------------
#---------Take n=50000, a=3.5 and 4.5-------------------------
X <- rnorm(n)
table(X>a)
```

```
##
##   FALSE    TRUE
## 499888     112
```

```r
mean((X>a))
```

```
## [1] 0.000224
```

```r
#----------Use Accept Reject--------------------------------
nsim <- 1000                #much smaller sample
X <- rexp(nsim,a)+a
mean(dnorm(X)/(a*exp(-a*(X-a))))
```

```
## [1] 0.0002321872
```

```r
#sd(dnorm(X)/(a*exp(-a*(X-a))))


#-----------Use Uniform-----------------------------------
U <- runif(nsim,0,1/a)
mean(dnorm(1/U)/(a*U^2))
```

```
## [1] 0.0002439268
```

```r
#sd(dnorm(1/U)/(a*U^2))
```